PROJECT REPORT

OF

UNDO-REDO-TEXT EDITOR



UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY:

Dharamveer Kushwaha | UID: 25BCS11049

Sarvesh Gupta | UID: 25BCS10169

Section/Group: 25BCS-705/B

Semester: 3rd

Program: B.E CSE Core

SUBMITTED TO:

Mr. Anmol Ray Sir (E19861)

CERTIFICATE

This is to certify that the team project entitled "**UNDO-REDO TEXT EDITOR**" has been prepared and submitted by **Dharamveer Kushwaha** (UID: 25BCS11049) and **Sarvsesh Gupta** (UID: 25BCS10169) as part of the academic requirements for the subject Advance Data Structure (Subject Code: 25CSH-202), Semester 3rd, Session July–December 2026.

The project demonstrates the use of stack and doubly linked list concepts through text editing, undo and redo operations, and an editing-history feature. The work is presented for academic evaluation.

We wish the student success in future academic and professional endeavours.

Signature of Project Guide

_____

Mr. Anmol Ray (E19861)

Technical Trainer

University Institute of Computer Science & Engineering

Chandigarh University

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Mr. Anmol Ray (E19861), Technical Trainer, for his valuable guidance and support throughout the completion of team project, “UNDO-REDO TEXT EDITOR.”

I am also thankful to Chandigarh University for providing the opportunity and resources required to complete this project as part of the subject Advance Data Structure (Subject Code: 25CSH-202), Semester 3rd, Session July–December 2026.

Finally, I would like to acknowledge everyone whose support and encouragement helped me complete this project successfully.

ABSTRACT

This project presents the design and implementation of an Undo-Redo Text Editor using C++ and Graphical User Interface (GUI) technology. The primary objective is to provide a simple text editor that allows users to enter, modify, and manage text efficiently.

The system uses stack data structures to perform Undo and Redo operations, enabling users to restore previous text states and recover undone changes. A doubly linked list is used to maintain the editing history, recording actions along with timestamps. The application also supports clearing text and opening and saving text files.

This project demonstrates the practical application of data structures in text editing and provides an understanding of stack operations, history management, and GUI development.

INDEX

1. Introduction
2. Problem Statement
3. Objectives
4. Data Structure Selection and Justification
5. Algorithm and Flowchart
6. Working Implementation (C++ Source Code)
7. Sample Input and Output
8. Time Complexity Analysis
9. Project Report and Demonstration
10. Conclusion
11. References

1. INTODUCTION

The Undo-Redo Text Editor is a simple and useful project developed to understand the practical application of data structures in text editing. While writing or editing text, users sometimes make mistakes or accidentally delete important content. This project helps users correct such mistakes by providing Undo and Redo operations.

The project includes features such as text editing, clearing text, opening and saving text files, and viewing editing history. The main concept behind the project is the use of Stack to manage previous and undone text states and a Doubly Linked List to maintain editing history in the C++ implementation. A graphical user interface makes the application easy to use and understand.

This project helped me understand how data structures can be used to solve real-life problems. It also improved my knowledge of C++ programming, logical thinking, and the development of user-friendly applications.

2. PROBLEM STATEMENT

While editing text, users may accidentally delete important content or make changes that they later want to reverse. Without a proper editing history, restoring previous text can become difficult and time-consuming.

The main problem addressed by this project is to develop a text editor that allows users to undo previous changes, redo undone changes, and view their editing history. The project uses Stack data structures to manage text states and a Doubly Linked List to maintain editing history in the C++ implementation. It also provides basic features such as clearing text and opening and saving text files.

The aim is to make text editing easier, more convenient, and reliable while demonstrating the practical use of data structures in solving real-world problems.

3. OBJECTIVES

The main objectives of the Undo-Redo Text Editor project are:

1. To develop a simple, user-friendly text editor with an interactive graphical interface.
2. To implement Undo and Redo operations using Stack to restore previous and undone text states.
3. To maintain editing history with timestamps using a Doubly Linked List in the C++ implementation.
4. To provide essential features such as text editing, clearing, opening, and saving text files.
5. To understand the practical application of data structures and improve programming and problem-solving skills through project development.

DATA STRUCTURE SELECTION AND JUSTIFICATION

The project uses the following data structures:

1. Stack: Two stacks, `undoStack` and `redoStack`, are used to store previous and undone text states. The Last In, First Out (LIFO) principle makes it easy to restore the most recent text state during Undo and Redo operations.

2. Doubly Linked List: A doubly linked list is used in the C++ implementation to maintain editing history. Each history entry stores information about an editing action, including its description and timestamp. It allows movement in both forward and backward directions through the history.

3. Justification: Stack is suitable for Undo and Redo because the most recent change must be reversed first. A doubly linked list is useful for maintaining and traversing editing history. These data structures make the application easier to manage and demonstrate the practical use of data structures in text editing.

Note: The C++ implementation uses `std::stack` for Undo and Redo and `std::list` for editing history. The web version uses JavaScript arrays for its stacks and history.

5. ALGORITHM AND FLOWCHART

5.1 Algorithm

Step 1: Start the application and initialize the text editor, Undo Stack, Redo Stack, and editing history.

Step 2: Allow the user to enter or modify text in the editor.

Step 3: Whenever the text changes, store the previous text state in the Undo Stack and clear the Redo Stack.

Step 4: When the user selects Undo, move the current text state to the Redo Stack and restore the previous state from the Undo Stack.

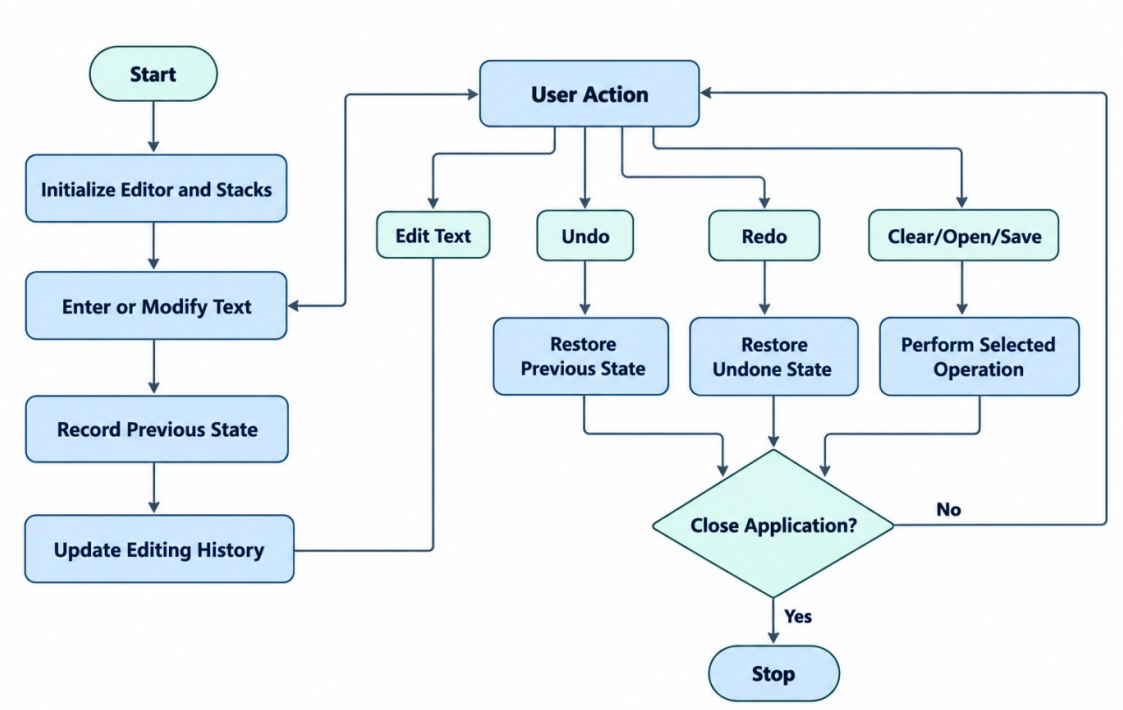
Step 5: When the user selects Redo, move the current text state to the Undo Stack and restore the state from the Redo Stack.

Step 6: Record editing actions in the history and allow the user to clear, open, or save text files.

Step 7: Repeat the operations until the user closes the application.

Step 8: Stop.

5.2 Flowchart



6. WORKING IMPLEMENTATION (C++ SOURCE CODE)

```cpp
#include <iostream>
#include <stack>
#include <vector>
#include <string>
using namespace std;

int main() {
    stack<string> undoStack, redoStack;
    vector<string> history;

    string text = "", newText;
    int choice;

    history.push_back(text);

    while (true) {
        cout << "\n\n--- UNDO-REDO TEXT EDITOR ---\n";
        cout << "Current Text: " << text << "\n";
        cout << "1. Edit Text\n";
        cout << "2. Undo\n";
        cout << "3. Redo\n";
        cout << "4. Show History\n";
        cout << "5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;
        cin.ignore();

        if (choice == 1) {
            undoStack.push(text);
            cout << "Enter new text: ";
            getline(cin, newText);
            text = newText;
            while (!redoStack.empty())
                redoStack.pop();
            history.push_back(text);
            cout << "Text updated!\n";
        }
        else if (choice == 2) {
            if (!undoStack.empty()) {
                redoStack.push(text);
                text = undoStack.top();
                undoStack.pop();
                history.push_back(text);
                cout << "Undo successful!\n";
            }
            else {
                cout << "Nothing to undo!\n";
            }
        }
        else if (choice == 3) {
            if (!redoStack.empty()) {
                undoStack.push(text);
                text = redoStack.top();
                redoStack.pop();
                history.push_back(text);
                cout << "Redo successful!\n";
            }
            else {
                cout << "Nothing to redo!\n";
            }
        }
        else if (choice == 4) {
            cout << "\n--- EDITING HISTORY ---\n";
            for (int i = 0; i < history.size(); i++) {
                cout << i + 1 << ". ";
                if (history[i].empty())
                    cout << "[Empty Text]";
                else
                    cout << history[i];
                cout << "\n";
            }
        }
        else if (choice == 5) {
            cout << "Editor closed!\n";
            break;
        }
        else {
            cout << "Invalid choice!\n";
        }
    }
    return 0;
}
```

7. SAMPLE INPUT AND OUTPUT

C:\Users\USER\Downloads\test undo.exe

```
--- UNDO-REDO TEXT EDITOR ---
Current Text:
1. Edit Text
2. Undo
3. Redo
4. Show History
5. Exit
Enter choice: 1
Enter new text: Dharamveer
Text updated!


--- UNDO-REDO TEXT EDITOR ---
Current Text: Dharamveer
1. Edit Text
2. Undo
3. Redo
4. Show History
5. Exit
Enter choice: 2
Undo successful!


--- UNDO-REDO TEXT EDITOR ---
Current Text:
1. Edit Text
2. Undo
3. Redo
4. Show History
5. Exit
Enter choice: 3
Redo successful!
```

```
--- UNDO-REDO TEXT EDITOR ---
Current Text: Dharamveer
1. Edit Text
2. Undo
3. Redo
4. Show History
5. Exit
Enter choice: 4

--- EDITING HISTORY ---
1. [Empty Text]
2. Dharamveer
3. [Empty Text]
4. Dharamveer


--- UNDO-REDO TEXT EDITOR ---
Current Text: Dharamveer
1. Edit Text
2. Undo
3. Redo
4. Show History
5. Exit
Enter choice: 5
Editor closed!

--------------------------------
Process exited after 66.98 seconds with return value 0
Press any key to continue . . .
```

8. TIME COMPLEXITY ANALYSIS

Time Complexity of Operations

Operation	Time Complexity	Explanation
Edit Text	O(L)	Stores and copies the text, where L is the text length.
Undo	O(L)	Restores the previous text state.
Redo	O(L)	Restores the undone text state.
Show History	O(H × L)	Displays all H history entries, each containing up to L characters.
Store History	O(L) amortized	Adds a text state to the history vector.

Explanation:

- **O(1):** Constant time for basic stack operations such as `push()`, `pop()`, and `top()`, excluding the cost of copying strings.
- **O(L):** Time may depend on the length of the text because the program stores and copies complete strings.
- **O(H × L):** Displaying the history may require visiting H entries and printing their text, with L representing the typical text length.

Here, **L** represents the length of the text, and **H** represents the number of entries in the history.

9. PROJECT REPORT AND DEMONSTRATION

The project report describes the design, implementation, and working of the Undo-Redo Text Editor. It explains the use of Stack and Vector data structures to perform editing operations and maintain editing history.

Project Repository:
GitHub: https://github.com/iamdharamveer/Undo-Redo-Text-Editor

Live Website Demonstration:
https://iamdharamveer.github.io/Undo-Redo-Text-Editor/

The project demonstration includes the following features:

1. **Edit Text:** Allows users to enter new text.
2. **Undo Operation:** Restores the previous text state.
3. **Redo Operation:** Restores the text state that was undone.
4. **Show History:** Displays the recorded text states in sequence.
5. **Exit:** Closes the console application.

The program is implemented in C++ and can be executed using Dev-C++. The GitHub repository contains the project files, while the live website provides an online demonstration of the text editor. During the demonstration, sample inputs are used to explain and test the Undo, Redo, and History operations.

10. CONCLUSION

The Undo-Redo Text Editor project demonstrates the practical use of data structures in C++ programming. It uses Stack to perform Undo and Redo operations and Vector to maintain editing history. The program allows users to edit text, restore previous text states, redo changes, and view the recorded history.

This project helped me understand the concepts of Stack, Vector, and their applications in solving real-world problems. It also improved my programming skills, logical thinking, and understanding of basic text editor functionality.

11. REFERENCES

1. **C++ Documentation:** https://en.cppreference.com/
2. **GeeksforGeeks – Stack Data Structure:** https://www.geeksforgeeks.org/stack-data-structure/
3. **GeeksforGeeks – Vector in C++ STL:** https://www.geeksforgeeks.org/vector-in-cpp-stl/
4. **GitHub Project Repository:** https://github.com/iamdharamveer/Undo-Redo-Text-Editor
5. **Live Project Demonstration:** https://iamdharamveer.github.io/Undo-Redo-Text-Editor/